

Computer Network

Practice: *File Transfer 1*

C&Lab.



Taewook Kim (TA)

Computer Communication Lab.
Dept. of Computer Engineering,
Chungbuk National University





Lecture Plan

Week	Date	Contents
1	9.1	-
2	9.8	Introduction & Ping
3	9.15	Inter-Process Communication
4	9.22	File Transfer 1
5	9.29	File Transfer 2
6	10.6	ARP
7	10.13	GARP, RARP
8	10.20	(middle test)
9	10.27	Internet Protocol Routing I & Static Router
10	11.3	Internet Protocol Routing II & Static Router
11	11.10	Internet Protocol Routing III & Static Router
12	11.17	Network Address Translation I
13	11.24	Network Address Translation II
14	12.1	Network Address Translation III
15	12.8	(final test)



Contents

- ◆ Peer to Peer
- ◆ Data Link Layer Communication
 - ❖ Transmission
 - ❖ Ethernet Protocol
 - ❖ Network API
- ◆ Byte Ordering
- ◆ Chatting



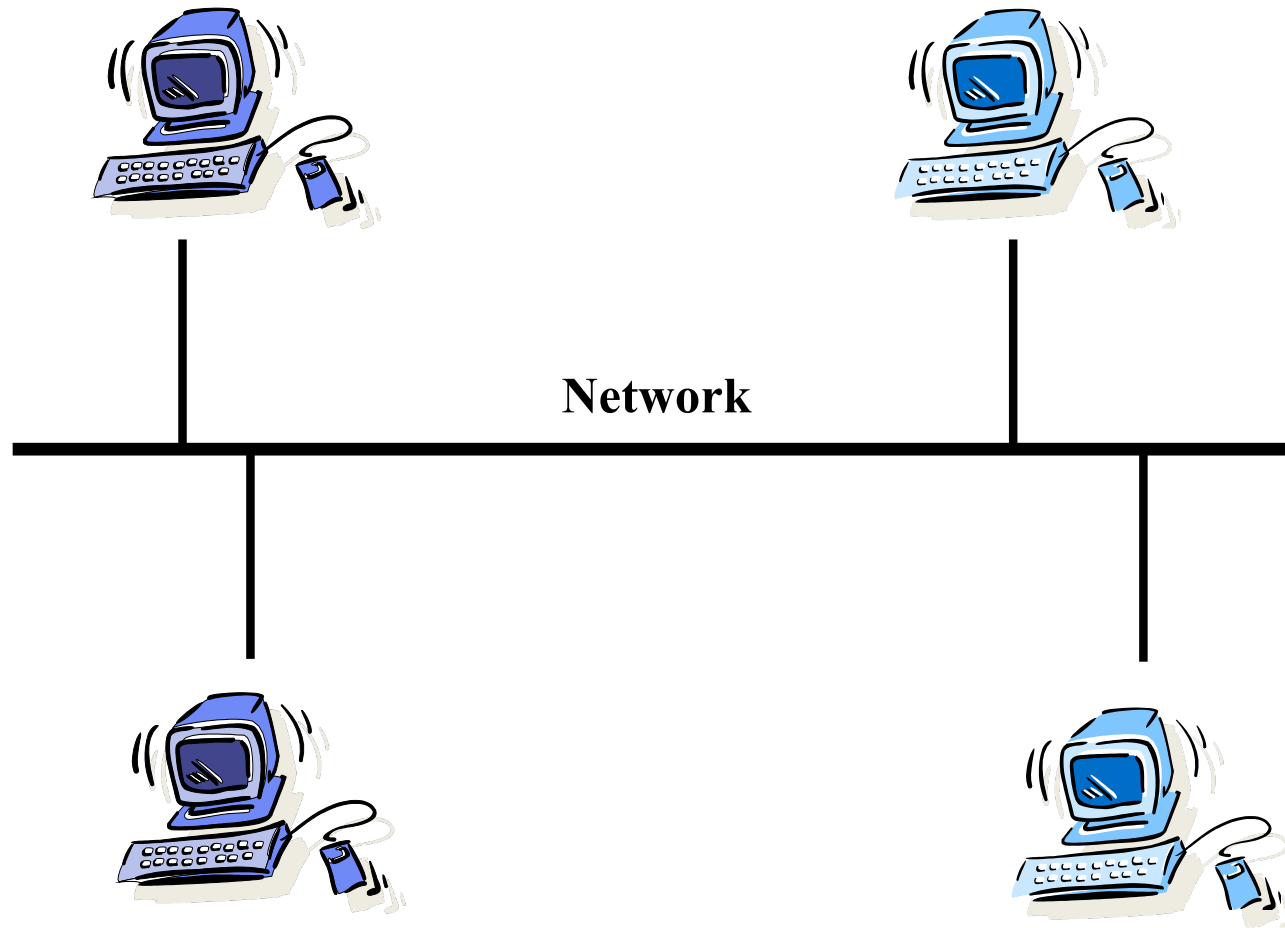
Peer to Peer

CLab.





Peer to Peer



Data Link Layer Communication

CLab.





Transmission

◆ Unicast

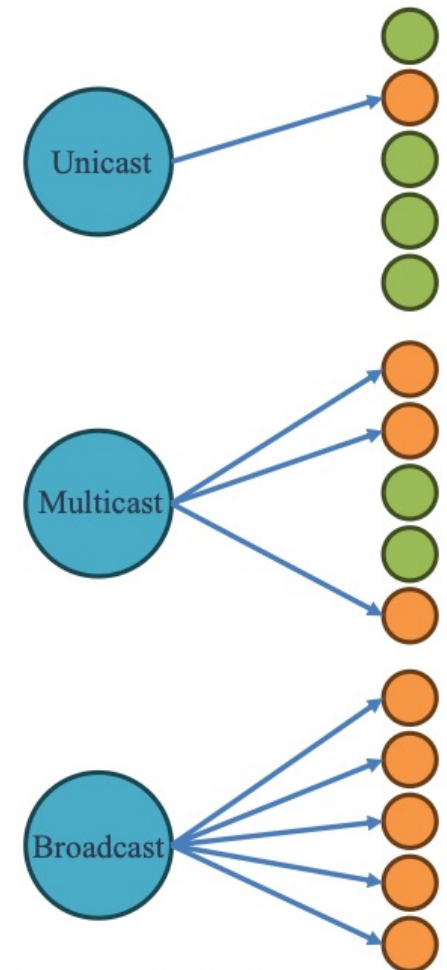
- ❖ Sending of messages to a single network destination identified by *a unique address*
- ❖ Point-to-point

◆ Multicast

- ❖ The delivery of a message or information to *a group of destination* computers *simultaneously* in a single transmission from the source creating *copies automatically* in other network elements
- ❖ One-to-many

◆ Broadcast

- ❖ The distribution of audio and video content to a dispersed audience via radio, television, or other
- ❖ FF:FF:FF:FF:FF:FF – Ethernet layer broadcast address





Ethernet Protocol

◆ 송신

- ❖ 상위 계층으로부터 데이터를 전달받으면 그 데이터를 프레임의 데이터에 저장
- ❖ 수신될 Ethernet 주소와 자신의 Ethernet 주소를 헤더에 저장
- ❖ 상위 계층의 종류에 따라서 헤더에 상위 프로토콜 타입을 저장 후 물리적 계층으로 Ethernet frame 전달

◆ 수신

- ❖ 하위 계층(physical layer)로부터 프레임을 받으면 상위로 보내야 하는지, 혹은 폐기해야 하는지 결정
- ❖ 상위 계층으로 보내는 기준
 - ◆ 목적지 Ethernet 주소가 브로드캐스트 주소(ff-ff-ff-ff-ff-ff)일 경우
 - ◆ 목적지 Ethernet 주소가 자신의 Ethernet 주소일 경우
- ❖ Ethernet 프레임 헤더 중에 16 비트 프로토콜 타입 필드를 보고 판단하여 상위 계층으로 전달
 - ◆ 0x0800: Internet Protocol (IP)
 - ◆ 0x0806: Address Resolution Protocol (ARP)
 - ◆ 0x0835: Reverse Address Resolution Protocol (RARP)

◆ Ethernet 프레임에서 최대 전송 단위 (MTU) : 1500bytes (Ethernet 헤더 제외)



Network API

◆ 요구사항

- ❖ 가능한 낮은 데이터 링크 계층까지 접근 가능
- ❖ Promiscuous Filter 가능 (Packet sniffing)
- ❖ Asynchronous Mode 로 동작 가능
- ❖ 사용자 정의 프로토콜을 사용할 수 있어야 함
- ❖ 사용자 계층에서 쉽게 접근 가능
- ❖ 네트워크 인터페이스의 정보 열람 가능
- ❖ 로컬 컴퓨터의 네트워크 설정 정보를 열람 가능
- ❖ 장치 드라이버 계층의 정보를 질의/설정 가능





Network API (cont.)

◆ Winsock (Windows Sockets API)

- ❖ 가장 대표적인 Internet Program 개발 API
- ❖ Network Layer까지 접근 가능
 - ◆ Raw Mode – Internet layer(IP)까지 지원
- ❖ Support Multicasting
- ❖ Overlapped Operation 지원
 - ◆ WSA_FLAG_OVERLAPPED flag
- ❖ Buffering management(Winsock 2.0)
- ❖ 다양한 프로토콜 지원
 - ◆ TCP, UDP, IP, DNS, ICMP, IGMP, IPX/SPX, ...etc
- ❖ 다양한 API 지원
 - ◆ Address conversion, byte ordering
- ❖ UNIX, Window 모두 지원





Network API (cont.)

◆ Packet Driver

- ❖ Data Link Layer까지 접근 가능
 - ◆ Read/Write의 기본 계층이 Ethernet Layer.
- ❖ Promiscuous operation 지원
 - ◆ Shared media의 모든 데이터 Read 가능
- ❖ Asynchronous operation 지원
- ❖ Kernel Driver 수준의 정보 Query/Setting 가능
 - ◆ H/W 주소 Query, Link Speed
- ❖ Protocol을 직접 구현 해야 함
- ❖ Buffering을 직접 구현해야 함
- ❖ Socket 보다 사용하기 어려움





Network API (cont.)

◆ Winsock vs. Packet Driver

	Winsock	Packet Driver
접근가능 계층	일반적으로 전송 계층 (제한적 접근) SOCK_RAW 모드 시 네트워크 계층까지 접근 (송신만 가능)	데이터 링크계층까지 모두 접근 가능
OVERLAPPED 방식	WSA_FLAG_OVERLAPPED flag를 통해 지원가능	지원가능
비동기 방식 송수신	지원가능	지원가능
드라이버 정보에 대한 질 의 및 설정 가능	API 를 통해 지원가능	지원가능
지원 프로토콜	전송계층까지 지원	직접 구현 해야 함
버퍼링	지원함	직접 구현 해야 함





Network API (cont.)

◆ WinPcap → Npcap

◆ Why the change?

- ❖ WinPcap's last release was 4.1.3 (2013); development has ended.
- ❖ It relies on the NDIS 5 driver model, which modern Windows no longer supports properly.
- ❖ Npcap uses NDIS 6 LWF, is digitally signed, and is actively maintained.
- ❖ ireshark and Nmap now install Npcap by default on Windows.

◆ What Npcap adds

- ❖ Loopback capture (traffic on the local machine)
- ❖ Raw 802.11 monitor mode for wireless adapters
- ❖ Optional restriction of driver access to Administrators
- ❖ WinPcap API-compatible mode

◆ For this lab

- ❖ Install Npcap with "WinPcap API-compatible Mode" checked.
- ❖ Existing code using wpcap.lib / packet.lib works unchanged.



Byte Ordering

CLab.





Byte Ordering

◆ Big Endian

- ❖ = Network Byte Order
- ❖ The most significant byte is on the left end of a word.
- ❖ Macintosh (Motorola) machines

◆ Little Endian

- ❖ = Host Byte Order
- ❖ The most significant byte is on the right end of a word.
- ❖ Intel-based machines





Byte Ordering (cont'd)

◆ Network Byte Order & Host Byte Order

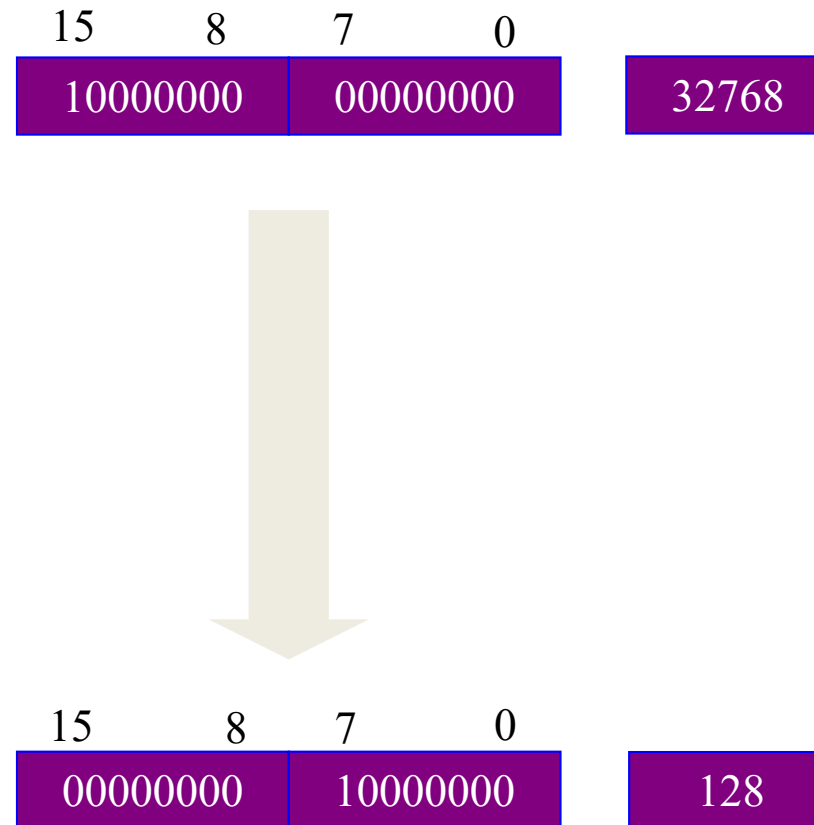
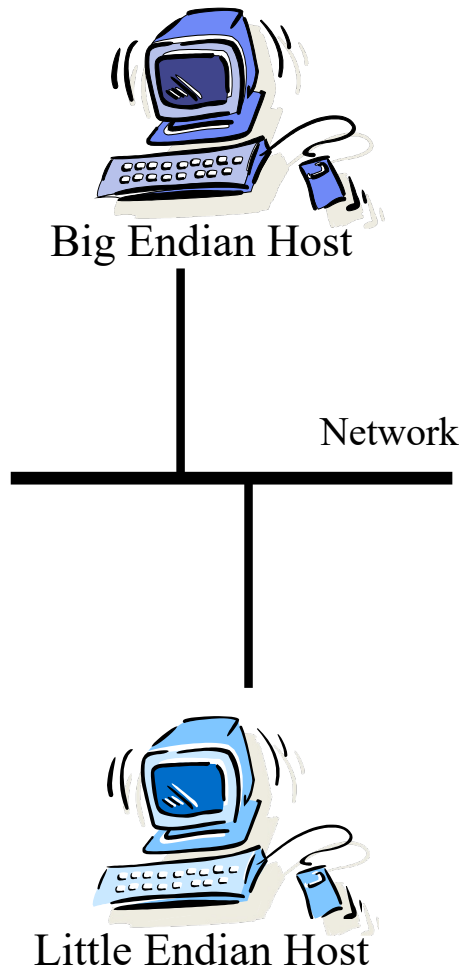
- ❖ So that machines with different byte order conventions can communicate, the Internet protocols specify a canonical byte order convention for data transmitted over the network.
- ❖ If you are encoding integer data in the messages sent through the socket, you should convert this to network byte order too.
- ❖ If you don't do this, your program may fail when running on or talking to other kinds of machines.





Byte Ordering (cont'd)

◆ When byte ordering is **NOT** applied





Byte Ordering (cont'd)

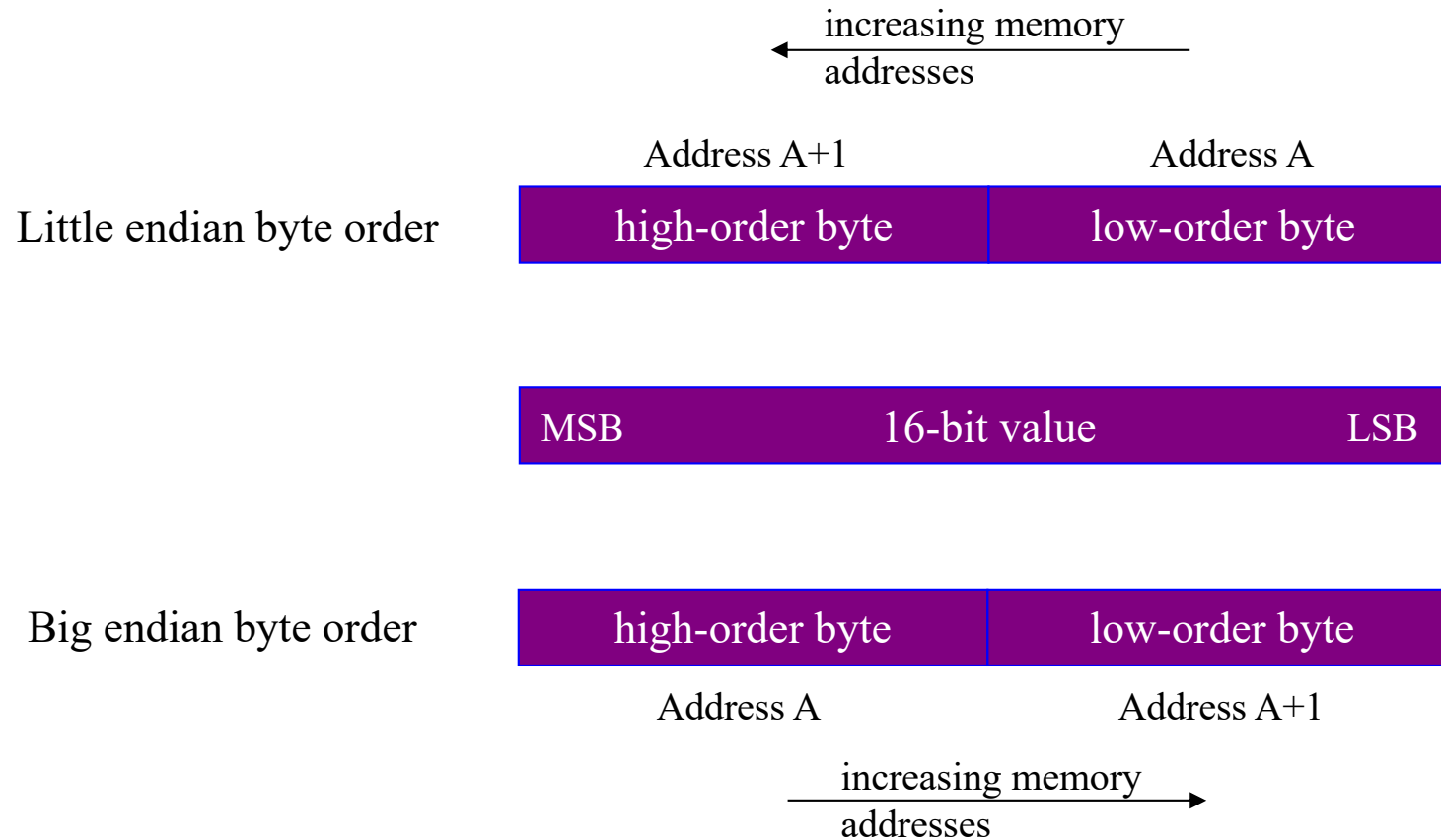


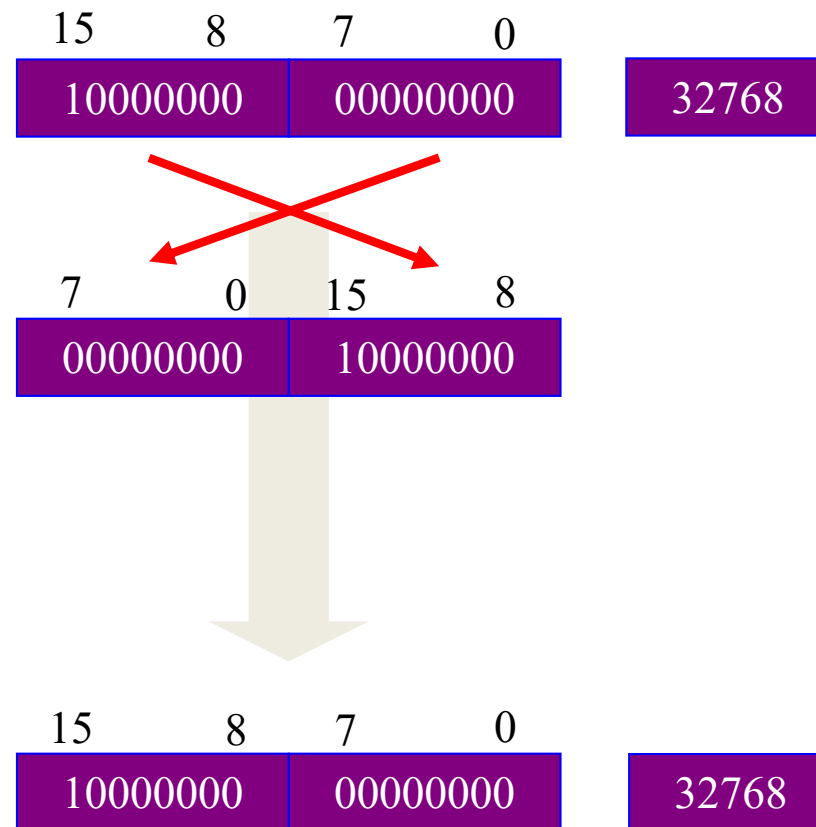
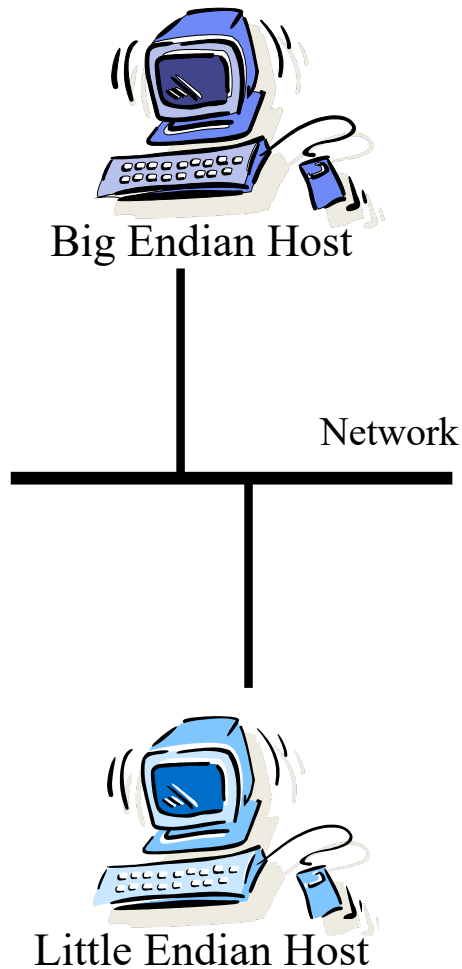
Fig. Little endian byte order and big endian byte order for 16-bit integer





Byte Ordering (cont'd)

◆ When byte ordering is applied





Byte Ordering (cont'd)

◆ 32-bit byte order

```
unsigned long int ntohl(unsigned long int x)
{
    return (
        ((x & 0x000000ffU) << 24) |
        ((x & 0x0000ff00U) << 8) |
        ((x & 0x00ff0000U) >> 8) |
        ((x & 0xff000000U) >> 24)
    );
}
```

- Little endian host must convert byte orders to 'network byte order' for sending data over the network.c





Byte Ordering (cont'd)

$(x \& 0x000000ffU) \ll 24$

x
0x000000ffU
&(Mask)
 $\ll$ (shift)

1	2	3	4
			4
4			

$(x \& 0x0000ff00U) \ll 8$

1	2	3	4
		3	
	3		

$(x \& 0x00ff0000U) \gg 8$

1	2	3	4
	2		
		2	

$(x \& 0xff000000U) \gg 24$

1	2	3	4
1			
			1

$((x \& 0x000000ffU) \ll 24) |$
 $((x \& 0x0000ff00U) \ll 8) |$
 $((x \& 0x00ff0000U) \gg 8) |$
 $((x \& 0xff000000U) \gg 24)$

4			
	3		
		2	
			1

Result

4	3	2	1
---	---	---	---



Chatting & File Transfer

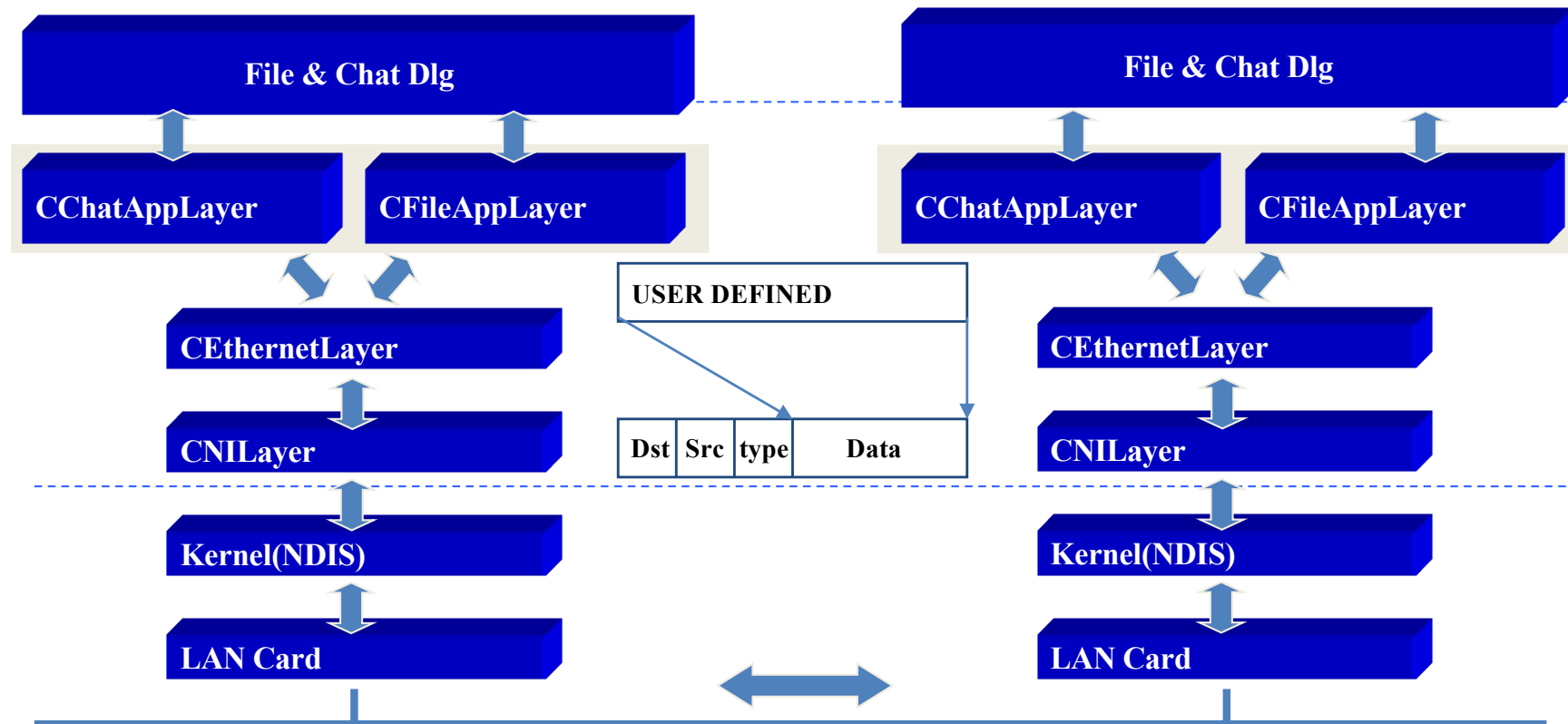
CLab.





Introduction

◆ Layer Model





Data Structure

◆ Ethernet Address

```
typedef struct _ETHERNET_ADDR
{
    union {
        struct { e0, e1, e2, e3, e4, e5; } s_un_byte ;
        unsigned char s_ether_addr[6] ;
    } S_un ;

#define addr0    S_un.s_un_byte.e0
#define addr1    S_un.s_un_byte.e1
#define addr2    S_un.s_un_byte.e2
#define addr3    S_un.s_un_byte.e3
#define addr4    S_un.s_un_byte.e4
#define addr5    S_un.s_un_byte.e5

#define addr     S_un.s_ether_addr

} ETHERNET_ADDR, *LPETHERNET_ADDR ;
```

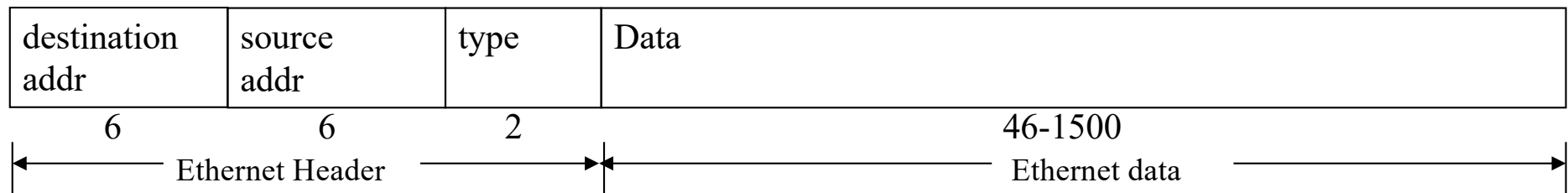




Data Structure (cont'd)

◆ Ethernet Protocol

```
typedef struct _ETHERNET
{
    ETHERNET_ADDR    enet_desaddr;    // 상대방 주소
    ETHERNET_ADDR    enet_srcaddr;    // 자기 주소
    unsigned short    enet_type;      // frame data type
    unsigned char     enet_data[ MAX_ETHERNET_DATA ];
} ETHERNET, *LPETHERNET;
```

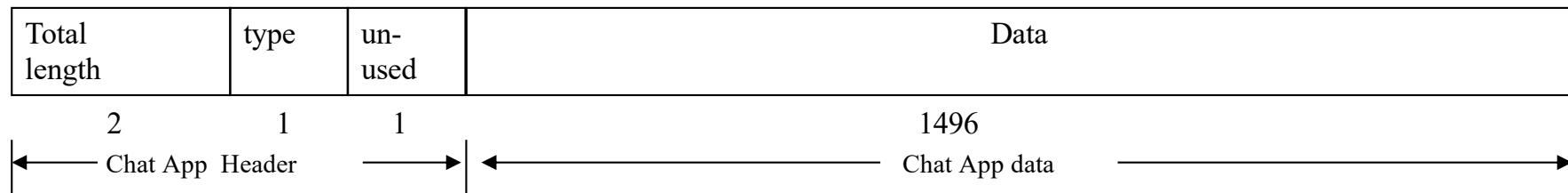




Data Structure (cont'd)

◆ Chat Application Protocol

```
typedef struct _CHAT_APP
{
    unsigned short    capp_totlen ;// 메시지 총 길이
    unsigned char     capp_type ; // 메시지 타입
    unsigned char     capp_unused ;      // 우선 사용 안함
    unsigned char     capp_data[ MAX_APP_DATA ] ;
} CHAT_APP, *LPCHAT_APP ;
```

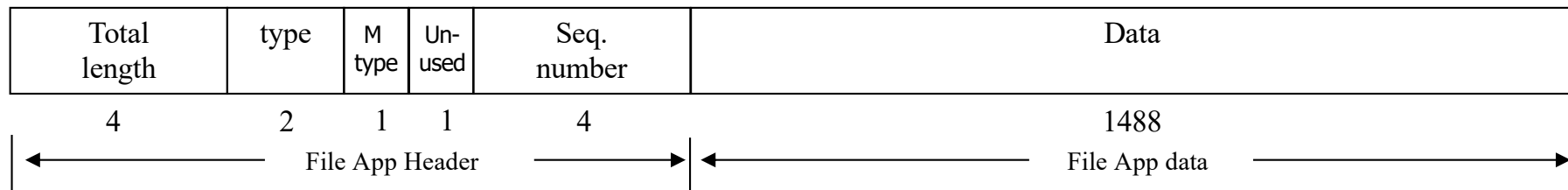




Data Structure (cont'd)

◆ File Transfer Application Protocol

```
typedef struct _FILE_APP
{
    unsigned long    fapp_totlen ; // 총 길이
    unsigned short   fapp_type ; // 데이터 타입
    unsigned char    faa_msg_type // 메시지 종류
    unsigned char    unused ;     // 사용 안함
    unsigned long    fapp_seq_num ; // fragmentation 순서
    unsigned char    fapp_data[ MAX_APP_DATA ] ;
} FILE_APP, *LPFILE_APP ;
```





CChatAppLayer

◆ CChatAppLayer

```
typedef struct _CHAT_APP
{
    unsigned short    capp_totlen ;           // 메시지 총 길이
    unsigned char     capp_type ;           // 메시지 타입
    unsigned char     capp_unused ;         // 우선 사용 안함
    unsigned char     capp_data[ MAX_APP_DATA ] ;
} CHAT_APP, *LPCHAT_APP ;
```

❖ Header size: $2 + 1 + 1 = 4$ bytes



◆ Chatting layer에서 사용되는 구조체

- ❖ totlen에 사용자가 입력한 문자열의 길이를 저장
- ❖ type에서 단편화에 대한 정보를 담을 수 있음
 - ◆ ex) 0x00 – 단편화 되지 않음, 0x01 – 단편화 첫부분, 0x02 – 단편화 중간, 등등..
- ❖ data에는 사용자가 입력한 문자열을 저장





CChatAppLayer (cont'd)

◆ 송신 과정

- ① 입력된 채팅 메시지의 길이를 측정
- ② 입력된 채팅 메시지의 길이가 1456bytes이하면, 단편화 없이 전송
- ③ 입력된 채팅 메시지의 길이가 1456bytes초과면, 단편화
- ④ 첫 번째 조각에 채팅 전체에 대한 정보를 담아서 전달
 - ◆ 전체 길이
- ⑤ 입력된 채팅 메시지를 1456bytes 단위로 단편화하여 전송
 - ◆ 이때 fapp_type을 통하여 처음, 중간, 끝을 구분해야 함

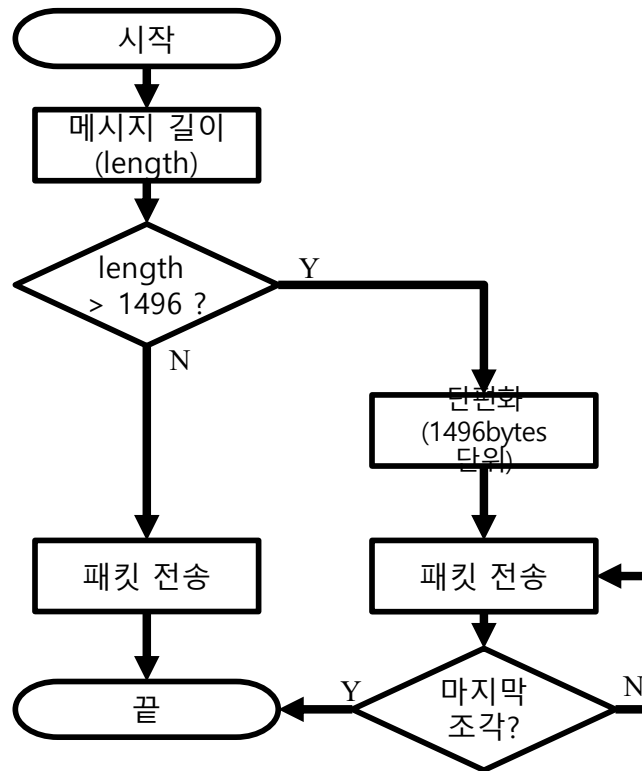
◆ 수신 과정

- ① 전달받은 첫 조각을 통해 단편화 여부를 확인
 - ◆ 전체 길이(fapp_totlen)를 통해 1456bytes 이상이라면 단편화가 되어있을 것임
- ② 단편화가 되어있지 않다면, 그대로 상위 레이어로 전송
- ③ 단편화가 되어있다면, buffer를 활용하여 채팅이 모두 전달될 때까지 덧붙임
- ④ 모든 조각을 전달받았다면, buffer에 쓰여진 내용을 상위 레이어로 전송
 - ◆ 처음 받은 조각의 fapp_totlen 정보와 수신된 데이터의 전체 길이를 비교하여 모든 조각을 전달받았는지 확인할 수 있음

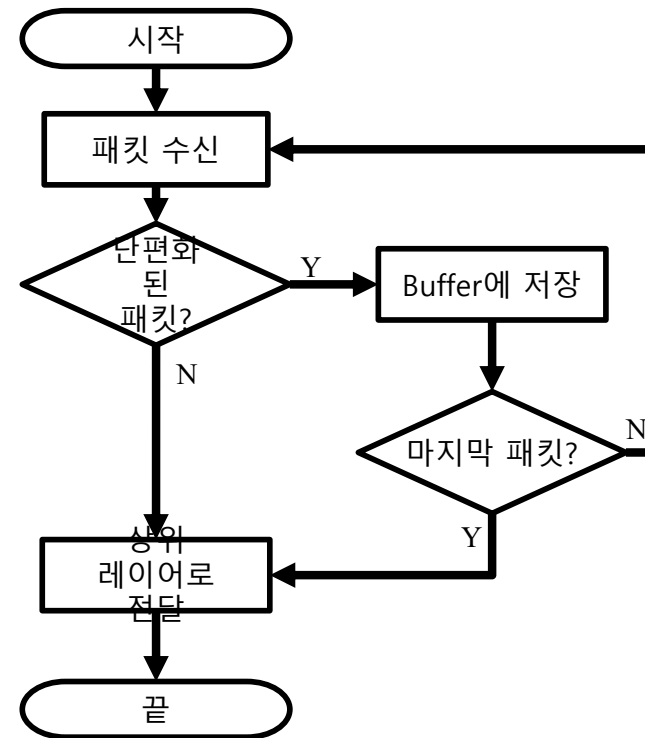




CChatAppLayer (cont'd)



송신



수신



Requirements Analysis

◆ Basic Design

- ❖ Chatting과 File 전송을 동시에 가능
 - ◆ Thread 구현
- ❖ Chatting Message와 File Size는 제한 없이 전송 가능
 - ◆ Fragmentation 가능
- ❖ 실제 Network Protocol (Ethernet Protocol)을 이용하여 Ethernet Frame을 송수신
 - ◆ Packet driver 이용하여 정보를 얻어 WinPcap을 이용하여 송수신
- ❖ Ethernet Frame을 수신
 - ◆ NILayer에서 수신만을 하기 위한 Thread 구현
- ❖ homework #01(IPC) 프로젝트에서 확장 구현
 - ◆ 새로 MFC 프로젝트 생성해도 됨
 - ◆ 반드시 IPC 프로젝트에서 확장 구현할 필요 없음





Requirements Analysis (cont'd)

- ◆ 모든 Layer는 CBaseLayer를 상속
- ◆ Layer 연결은 LinkManager 클래스의 함수 사용





Requirements Analysis (cont'd)

◆ Chatting Application Layer (class CChatAppLayer)

❖ Ethernet Frame Number of Ethernet Layer : 0x2080 (user defined)

❖ Chatting Message에 대해서 Fragmentation을 함으로써 길이에 제한 없이 전송하도록 구현

❖ Fragmentation

◆ 전송 시

- 데이터를 일정 크기로 잘라서 하나씩 전송
- capp_type에 메시지 데이터가 첫 부분(0x00)인지, 중간 부분 (0x01) 인지, 마지막 부분 (0x02) 인지 구분하여 데이터를 전송
 - + 예) capp_type: 0x00 → 데이터 길이

◆ 수신 시

- 데이터가 첫 부분 일 경우 그 크기만큼 버퍼 할당
- 데이터가 중간부분인 경우 계속 버퍼에 저장
- 데이터가 마지막 부분일 경우 버퍼에 저장된 메시지 전체를 Dialog 객체에게 전달 (상위 레이어로 전달)





Requirements Analysis (cont'd)

◆ File Transfer Application Layer (class CFileAppLayer)

- ❖ Ethernet Frame Number of Ethernet Layer : 0x2090 (user defined)
- ❖ File에 대해서 Fragmentation을 함으로써 size에 제한 없이 전송하도록 구현
- ❖ Fragmentation

◆ 전송 시

- 파일명을 얻고, 파일을 열어서 버퍼에 저장
- 첫 부분 : 파일에 대한 정보를 전송 (이름, 크기, 타입 등), fapptype = 0x00
- 중간부분 : 파일의 데이터 전송 (Fragmentation적용), fapptype = 0x01
- 마지막 부분 : 모든 데이터가 다 전송되고 마지막이라는 메시지, fapptype = 0x02
- 모든 데이터 전송이 완료된 후 초기상태로 전환

◆ 수신 시

- 첫 부분(fapptype = 0x00) 일 경우
 - + 그 파일 정보를 받아서 파일 생성, 생성시 파일명으로 파일 크기만큼의 파일을 저장할 공간 확보
- 중간부분 (fapptype = 0x01) 일 경우
 - + 생성된 파일에 데이터를 덧붙인다
- 마지막 부분 (fapptype = 0x02) 일 경우
 - + 생성된 파일의 크기 및 순서를 고려 잘 전송 받았는지 확인하고 파일을 닫음





Requirements Analysis (cont'd)

◆ Ethernet Layer(class CEthernetLayer)

- ❖ 수신된 프레임 중에서 destination address가 자기 것이 아니면 discard한다.
- ❖ 수신된 프레임 중에서 source address가 자기 것이라면 discard한다.
- ❖ 그 밖의 구현에 필요한 member method는 추가
- ❖ 헤더를 제외한 데이터 부분은 Chatting App Layer or File App Layer로 전송
 - ◆ Frame Type가 0x2080이면 Chat App Layer에게, 0x2090이면 File App Layer로 전송
 - ◆ Frame Type은 반드시 0x2080 또는 0x2090일 필요 없음, 구분만 하면 됨
 - ◆ 어떤 Frame Type 번호를 사용했는 지, 보고서에 명시만 하기 바람

◆ Network Interface Layer (class CNILayer)

- ❖ WinPcap을 이용하여 기본적인 packet 송수신 operation 구현한 class
- ❖ Adapter와 상위 Layer간의 데이터 송수신의 중간자 역할을 담당
- ❖ Packet32.h를 이용하여 Mac 정보를 얻어온다.
 - ◆ 코드로 구현 가능
 - ◆ 코드 구현이 어려우면, MAC 주소를 코드로 직접 입력해도 됨
 - ◆ 보고서에 명시





Requirements Analysis (cont'd)

◆ Thread 구현

❖ UINT ReadingThread(LPDWORD lpdwParam)

◆ NILayer

◆ 수신된 Ethernet Frame을 받아서 상위 레이어로 전달하는 역할을 담당

❖ UINT FileTransferThread(LPDWORD lpdwParam)

◆ NILayer or FileAppLayer

◆ Chatting과 File Transfer를 동시에 가능하게 하기 위해 Thread를 구현

◆ Fragmentation 기능 포함

❖ 이 외에 Thread가 필요한 Layer에서 구현 가능

❖ 보고서에 반드시 명시

◆ 어느 Layer에 Thread를 사용했는 지

◆ 왜 사용했는 지





User Interface

◆ Chatting

- ❖ Chatting Message가 출력될 Window
- ❖ 보내질 Chat message가 입력될 Window
- ❖ Send Button

◆ File Transfer

- ❖ File Name을 불러올 Button
- ❖ File Name을 화면에 출력할 Window
- ❖ 전송 상태를 보여줄 Window
- ❖ Send Button

◆ Address

- ❖ Source Ethernet Address 보여줄 Window
- ❖ Destination Ethernet Address 입력할 Window
- ❖ 설정 Button





User Interface (cont'd)

◆ Example





Assignment – 4

VMware Workstation 16 Player (Non-commercial use only)

Player | [Icons] | [Buttons]

프로세스: [7268] Filetransfer2019

PC2

PC1

Frame 53097: 877.738323 VMware_c0:00:08 VMware_5d:8b:a9
53098 878.197646 192.168.232.130 192.168.232.2
53099 878.207796 192.168.232.2 192.168.232.130

> Frame 53096: 41 bytes on wire (328 bits), 41 bytes captured
> Ethernet II, Src: VMware_c0:00:08, Dst: VMware_5d:8b:a9
Data (27 bytes)
Data: cdcddcd14000068656
[Length: 27]

Source Address: [00:50:56:c0:00:08:00:0c:29:5d:8b:a9] hellow i'm program 1
Destination Address: [00:50:56:c0:00:08]

Source Address: [00:50:56:c0:00:08] hellow i'm program
Destination Address: [00:0c:29:5d:8b:a9]

메세지 전송 | 파일 추가 | 파일 전송 | 재설정(R)

```
HANDLE hFile;  
DWORD dwRead = 0;  
char* pszBuf;  
BOOL bResult;  
  
int progress_valu  
int i = 0, tot_se  
CString fileName;  
  
static unsigned c  
  
hFile = NULL;  
pszBuf = NULL;  
  
if (bFILE == TRUE  
{  
    // 핸들러에 파  
    hFile = Creat  
    if (hFile ==  
    {  
        AfxMessag  
        ((CFiletr  
        return FA  
    }  
  
    // 파일경로에서 파일명만 가져옴  
    fileName = fapp_hdr->m_FilePath;  
    fileName = fileName.Right(fileName.GetLength() - fileName.ReverseFind('\\') - 1);  
  
    // 파일 전체 크기(byte)를 가져옴  
    dwFileSize = GetFileSize(hFile, 0);
```



Assignment – 4 (cont'd)

◆ Wireshark

Capturing from VMware Network Adapter VMnet8

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter: <Ctrl-F>

No.	Time	Source	Destination	Protocol	Length	Info
53092	876.864619	192.168.232.130	20.62.190.186	TCP	54	49819 → 443 [FIN, ACK] Seq=1936 Ack=19900 Win=63502 Len=0
53093	876.864951	20.62.190.186	192.168.232.130	TCP	54	443 → 49819 [ACK] Seq=19900 Ack=1937 Win=64239 Len=0
53094	877.117024	20.62.190.186	192.168.232.130	TCP	54	443 → 49819 [FIN, PSH, ACK] Seq=19900 Ack=1937 Win=64239 Len=0
53095	877.117396	192.168.232.130	20.62.190.186	TCP	54	49819 → 443 [ACK] Seq=1937 Ack=19901 Win=63502 Len=0
53096	877.738323	VMware_c0:00:08	VMware_5d:8b:a9	41 Ethernet II	41	Ethernet II
53098	878.107646	192.168.232.130	192.168.232.2	DNS	80	Standard query 0x336b A v10.events.data.microsoft.com
53099	878.207796	192.168.232.2	192.168.232.130	DNS	105	Standard query response 0xa36b A v10.events.data.microsoft.com A 20.42.65.88
53100	878.209533	192.168.232.130	20.42.65.88	TCP	66	49820 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM=1
53101	878.471263	20.42.65.88	192.168.232.130	TCP	58	443 → 49820 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
53102	878.471701	192.168.232.130	20.42.65.88	TCP	54	49820 → 443 [ACK] Seq=1 Ack=1 Win=64240 Len=0

Frame 53096: 41 bytes on wire (328 bits), 41 bytes captured (328 bits) on interface \Device\NPF_{44E6B885-38C6-4494-A0A1-EEECDA42CCA1}, id 0
Ethernet II, Src: VMware_c0:00:08 (00:50:56:c0:00:08), Dst: VMware_5d:8b:a9 (00:0c:29:5d:8b:a9)
Data (27 bytes)
Data: cdcddcd14000068656c6cf772069606d2070726f6772616d2031
[Length: 27]

0000 00 0c 29 5d 8b a9 00 50 56 c0 00 08 00 20 cd cd ...].P.V...
0010 cd cd 14 00 00 68 65 6c 6c 6f 77 20 69 60 6d 20hel low i'm
0020 70 72 6f 67 72 61 6d 20 31 program 1

VMware Network Adapter VMnet8: <live capture in progress> | Packets: 58498 | Displayed: 58498 (100.0%) | Profile: Default



Assignment – 4 (cont'd)

◆ 전송 시나리오 (참고)

- ❖ 두 대의 PC에서 각각 프로그램을 실행
- ❖ 두 대의 PC는 네트워크로 연결
- ❖ PC 1의 프로그램
 - ◆ 1. 전송할 파일 선택 or 1. 전송할 텍스트 입력
 - ◆ 2. 전송 버튼을 클릭
 - ◆ 3. 레이어 아키텍처에 의해서 Encapsulated Packet이 만들어짐 (Ethernet frame)
 - ◆ 4. NILayer의 Pcap 라이브러리로 작성된 코드에 의해 packet은 네트워크로 전송됨
- ❖ PC 2의 프로그램에서 PC1으로부터 Ethernet frame을 수신
 - ◆ 1. 하위 레이어부터 수신된 프로토콜이 PC 1로부터 보내진 패킷인지 검사
 - 아니라면, discard(버림)
 - ◆ 2. 맞으면, 레이어 아키텍처에 의해서 Demultiplexing을 통해 header를 제외한 data 부분을 상위 레이어로 전달
 - Header는 매 레이어마다 receive 함수에서 frame에 대해 나에게 온 것이 맞는 검사할 때 쓰임
 - ◆ 3. 최상위 레이어 (*Dlg.cpp)에 전달 될 때 까지 1번과 2번 과정을 반복함
 - ◆ 4. 전달된 결과 확인
 - 파일인 경우, 채팅&파일전송 프로그램이 위치한 곳에 설치됨 (기본 저장 경로 사용 시)
 - 텍스트인 경우, 채팅&파일전송 프로그램 화면에서 채팅창(CListbox)에 표기 됨
 - + 표기 방법은 IPC에서 채팅 표기 방법과 동일





Assignment – 4 (cont'd)

◆ 보고서 내용 (조별)

❖ 실습 개요

- ◆ 실습 일시 및 장소
- ◆ 실습 목적
- ◆ 실습 시나리오

❖ 프로토콜 스택 (해당 프로그램)

- ◆ 구조 설명 (그림, 텍스트)
- ◆ 각 프로토콜(layer)의 역할 등

❖ 구현 설명

- ◆ 사진 필수 (함수 역할 및 함수 구현 설명)

❖ 실행 결과

- ◆ 프로그램 결과 화면
- ◆ 패킷 캡처 화면

- 방법은 E-campus에 올라와 있는 Appendix-Wireshark 참조





Assignment – 4 (cont'd)

◆ 파일 업로드 시 주의!! (MFC경우)

- ❖ 업로드 파일의 크기가 커서 업로드가 되지 않은 경우가 있음
- ❖ 프로젝트 폴더에서 다음을 제거하고 압축하기 바람
 - ◆ Debug 폴더 제거
 - ◆ ipch 폴더 제거
 - ◆ [project name].sdf 파일 제거 (솔루션 파일, 용량 제일 많이 차지하는 파일)

◆ 과제 제출일

- ❖ 최종 제출: 10월 1일 00시까지
- ❖ E-Campus 참고
- ❖ 조 별 대표 1명만 제출





Notice

- ◆ 과제 수행 전에 driver development kit와 Packet Protocol 을 설치
 - ❖ Wireshark (패킷 캡처프로그램)를 설치 시, npcap 및 usbpcap 설치가 용이
 - ❖ E-Campus → Appendix – Packet Driver 참조





Notice (cont'd)

◆ 보고서 작성에 소홀하지 않도록 주의

- ❖ 자신의 조에서 구현한 것은 최대한 표현
- ❖ 보고서 양식은 **P.42**를 참고
- ❖ 분량은 제한 없음
- ❖ 구현 설명 시
 - ◆ 보고서에 코드 작성 시, 자신들이 구현한 부분에서 중요한 부분만 추가하고 설명 첨부
 - 각 Layer에서 사용된 프로토콜 header의 field에 대해서 언급
 - ◆ 실행화면은 필요한 화면만 캡처하여 보고서에 붙여 넣고 설명 첨부
- ❖ 붉은 글씨에 유의하여 보고서를 작성 바람
 - ◆ 붉은 글씨의 내용만 보고서에 작성하라는 것이 아님
 - ◆ 붉은 글씨의 내용도 보고서에 작성하라는 것임 (착각 금물)





Notice (cont'd)

◆ 실습 환경 준비물

- ❖ Npcap — 패킷 드라이버
- ❖ Npcap SDK — 빌드용 헤더 / 라이브러리
- ❖ LAN 케이블 — PC 2대 직접 연결
 - ◆ LAN 케이블 없는 팀은 연구실 방문 후 제작하여 수령
- ❖ USB-이더넷 젠더 (해당자)
 - ◆ 노트북에 RJ45 포트가 없는 경우
 - ◆ 연구실에서 Type-A 팀당 1개 대여
 - ◆ Windows가 일반 랜카드로 인식하므로 추가 설정 불필요





Assignment - 5

- ◆ 개별 과제
- ◆ Selective Repeat ARQ / Stop and Wait ARQ 각각의 특징 및 차이점 조사
- ◆ 양식 자유



Q&A

CLab.

